

NOVA RETAIL

ENTERPRISE DATA ARCHITECTURE & TECHNICAL DOCUMENTATION

End-to-End Modern Data Stack Delivery | Medallion Pipeline & BI Audit

Data Engineer: Alicia Casanueva

Submission Date: September 2026

Code Repository: https://github.com/Alicasanueva/nova_retail_dbt

Executive Summary & Architecture Flow

The Nova Retail project moves the company from fragmented, manually managed local files (Sales CSV transactions, Web Reviews JSON dumps and Finance Excel targets) to a Medallion Architecture (Bronze → Silver → Gold) on Snowflake. KNIME ingests the raw files, dbt Cloud transforms and tests the data, and an executive Power BI dashboard is built strictly on top of the Gold star schema, giving the C-level a single source of truth.



Figure 1. End-to-end pipeline lineage (Local files → KNIME → Snowflake Bronze → dbt Silver → dbt Gold → Power BI)

Results at a glance

Metric	Value	Source
Sales transactions ingested	100 rows: 100 distinct orders, 10 customers, 10 products, 4 categories	raw_sales_transactions.csv
	60 reviews; 48 valid ratings, 12 invalid ratings set to NULL	web_reviews.json
Finance targets	16 rows × 12 months → 192 rows after unpivot; annual target €3,768,960	finance_targets_2023.csv
Orders in the dashboard	100	Power BI
Total net revenue (EUR)	€59,600 (profit €17.9K at the 30% margin assumed by the brief)	Power BI
Gold star schema	5 tables: DIM_DATE, DIM_CUSTOMERS, DIM_PRODUCTS, FCT_SALES, FCT_MONTHLY_TARGETS	Snowflake GOLD

1. Technology Choices & Alternatives Considered

To replace Nova Retail's legacy Excel-based operations with an enterprise-grade data platform, a Modern Data Stack (MDS) was selected based on strict decoupling principles. Each component is assigned a single, specialized responsibility within the Medallion Architecture (BRONZE, SILVER, GOLD), ensuring high scalability, data governance, and operational reliability.

1.1 Responsibility of each tool and why it was chosen

Layer	Tool	Responsibility	Why this tool
Ingestion (local files → Bronze)	KNIME Analytics Platform	Read local CSV/JSON files, flatten the nested JSON and write the raw data to Snowflake without applying business logic.	Visual, low-code ETL engine ideal for hybrid file ingestion. Provides native JSON Path and Ungroup nodes for semi-structured payloads, robust Snowflake connectors, and zero-licensing overhead for desktop batch execution.

Layer	Tool	Responsibility	Why this tool
Storage & compute	Snowflake	Host Bronze, Silver and Gold and run every transformation.	SQL-first architecture with native VARIANT support for semi-structured JSON payloads and TRY_* conversion functions for robust data quality handling. Configured with a dedicated XSMALL warehouse featuring a 60-second auto-suspend threshold for optimal compute cost management. Serves as the single governed source of truth across Silver and Gold layers. Alternative platforms like Databricks (Spark/ML) were excluded to avoid cluster startup latency and unnecessary infrastructure overhead for the current workload scale.
Transformation & testing	dbt Cloud	Bronze → Silver → Gold SQL models, automated tests, documentation, lineage and scheduled jobs.	Industry standard for SQL-first transformation, providing automated testing, DAG lineage graph generation, data documentation, and managed Cloud orchestration without requiring local infrastructure.
Visualisation	Power BI Desktop	Semantic model, DAX measures and the executive dashboard.	Enterprise BI standard with powerful DAX time-intelligence functions (DATESMTD, DATESYTD), native Field Parameters for dynamic slice-and-dice, and direct integration with Snowflake Gold star schemas.

1.2 Why not a single tool, or fewer tools?

Alternative	What would be lost
Only KNIME (ingest, transform and serve)	Business logic would live inside workflow nodes: hard to review and version, no automated data tests, no lineage documentation, and the transformations would run on the local machine instead of the warehouse. Power BI would read a KNIME output instead of a governed Gold layer.
Only Snowflake	Loading local unstructured/semi-structured files directly into Snowflake requires custom script-based staging (PUT/COPY INTO) without visual lineage or user-friendly parsing for business operators.
KNIME + Snowflake, without dbt	Technically feasible, but transformations become opaque SQL scripts or complex KNIME database nodes. Lacks automated data testing frameworks, version-controlled DAG lineage, and environment segregation.
Snowflake + dbt, without KNIME	Snowflake and dbt are optimized for in-warehouse processing, requiring an edge ingestion connector to parse and stream heterogeneous local files (CSV/JSON) into the landing schema.
Transforming inside Power BI (Power Query / DAX)	Business logic becomes locked inside a single report file (.pbix), preventing reusability across enterprise applications, hindering automated testing, and violating the single-source-of-truth governance model.

1.3 Architecture trade-offs & mitigations

- **More moving parts.** Four tools instead of one. Mitigation: each has a single, documented responsibility and the interfaces are the Snowflake schemas.
- **KNIME Analytics Platform** is a desktop-based open-source tool that lacks native cloud scheduling capabilities without a paid KNIME Business Hub license. Mitigation: Automated ingestion was achieved locally by creating a Windows Batch script (run_knime_ingestion.bat) that triggers KNIME in headless CLI batch mode (org.knime.product.KNIME_BATCH_APPLICATION) with the -reset flag, scheduled via Windows Task Scheduler to run daily at 06:00 AM.").

2. Storage Infrastructure & Snowflake Setup

Compute is isolated in a dedicated XSMALL warehouse (DBT_WH, auto-suspend after 60 seconds) and data is separated by maturity into three schemas: BRONZE (raw, immutable), SILVER (cleaned, typed, normalised) and GOLD (star schema for Power BI).

During the initial delivery phase, a unified transformation role (DBT_ROLE) with broad database privileges was used for simplicity. In a production environment, strict Role-Based Access Control (RBAC) should separate responsibilities using dedicated loader (LOADER_ROLE), transformation (DBT_ROLE), and read-only BI (BI_READER_ROLE) roles.

```
-- 1. Compute warehouse, database and schemas
CREATE WAREHOUSE IF NOT EXISTS DBT_WH
  WITH WAREHOUSE_SIZE = 'XSMALL' AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE INITIALLY_SUSPENDED = TRUE;
CREATE DATABASE IF NOT EXISTS NOVA_RETAIL_DB;
CREATE SCHEMA IF NOT EXISTS NOVA_RETAIL_DB.BRONZE;
CREATE SCHEMA IF NOT EXISTS NOVA_RETAIL_DB.SILVER;
CREATE SCHEMA IF NOT EXISTS NOVA_RETAIL_DB.GOLD;

-- 2. Core transformation role (unified development scope)
CREATE ROLE IF NOT EXISTS DBT_ROLE;
GRANT USAGE ON WAREHOUSE DBT_WH TO ROLE DBT_ROLE;
GRANT ALL ON DATABASE NOVA_RETAIL_DB TO ROLE DBT_ROLE;
GRANT ALL ON ALL SCHEMAS IN DATABASE NOVA_RETAIL_DB TO ROLE DBT_ROLE;
GRANT ROLE DBT_ROLE TO USER ALICIASANUEVA;

-- 3. Bronze raw tables (every column STRING / VARIANT + load audit column)
USE DATABASE NOVA_RETAIL_DB; USE SCHEMA BRONZE;
CREATE OR REPLACE TABLE RAW_SALES_TRANSACTIONS (
  order_id STRING, transaction_date STRING, customer_info STRING,
  product_id STRING, product_category STRING, price STRING,
  qty STRING, discount_pct STRING,
  _loaded_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP());
CREATE OR REPLACE TABLE RAW_WEB_REVIEWS (
  json_data VARIANT,
  _loaded_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP());
CREATE OR REPLACE TABLE RAW_FINANCE_TARGETS (
  region STRING, category STRING,
  jan_23 STRING, feb_23 STRING, mar_23 STRING, apr_23 STRING,
  may_23 STRING, jun_23 STRING, jul_23 STRING, aug_23 STRING,
  sep_23 STRING, oct_23 STRING, nov_23 STRING, dec_23 STRING,
  _loaded_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP());

-- 4. Reference table for currency conversion (loaded by hand, not by KNIME)
CREATE OR REPLACE TABLE RAW_EXCHANGE_RATES (
  RATE_DATE DATE, CURRENCY_CODE VARCHAR(3), EXCHANGE_RATE_TO_EUR NUMBER(10,4));
INSERT INTO RAW_EXCHANGE_RATES VALUES
  ('2023-01-01', 'USD', 0.9200), ('2023-01-01', 'GBP', 1.1500), ('2023-01-01', 'EUR', 1.0000);
```

Snowflake DDL executed for the project (full script to be uploaded to the repository as /snowflake/01_setup.sql).

Exchange rates: what they are and what they are not

RAW_EXCHANGE_RATES holds one static rate per currency, dated 2023-01-01 (USD → EUR 0.92, GBP → EUR 1.15). It is an assumption of the project, not part of the source files, and the same rate is applied to every 2023 transaction. It is loaded with an INSERT, not by KNIME.

3. Source Profiling, Ingestion & Data Quality

3.1 Source profile

File	Volume	Grain	Notes
raw_sales_transactions.csv	100 rows, 8 columns	1 row = 1 order (100 distinct order_id, one product per order)	10 customers, 10 products, 4 categories. Each product has one fixed category, price and currency.
web_reviews.json	60 reviews	1 review; nested user{name, email, location{country, city}}	10 reviewers (the same 10 emails as the sales customers); every product_ref exists in sales.
finance_targets_2023.csv	16 rows × 12 months	Region × Category (wide format)	4 regions × 4 categories. Annual total €3,768,960; the currency is not stated in the file.

3.2 KNIME ingestion workflow

Three branches load the three sources into Snowflake BRONZE: (1) a CSV Reader feeding a DB Writer for the sales file; (2) a JSON Reader → JSON Path → Ungroup → JSON Path → Column Filter chain for the nested reviews, feeding a DB Writer; (3) a CSV Reader feeding a DB Writer for the finance targets. One Snowflake DB Connector supplies the connection to the three DB Writer nodes. The Bronze tables define every column as STRING (or VARIANT), so no source value can be rejected at load time.

The ingestion pipeline is designed for automated execution in Batch CLI mode via Windows Task Scheduler. Landing files are read using standard CSV Reader and JSON Reader nodes, flattening nested review payloads before writing to Snowflake BRONZE.

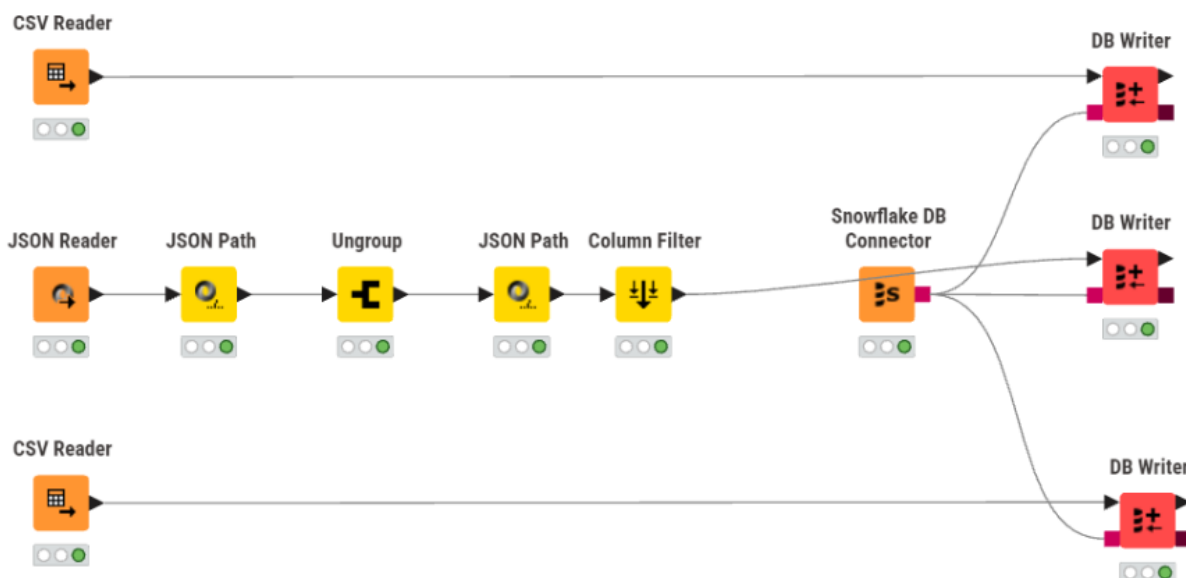


Figure 2. KNIME ingestion workflow (Local files → Snowflake BRONZE).

3.3 Data quality issue log

Counts below were measured on the three raw files provided (100 sales rows, 60 reviews, 16 target rows).

ID	Source · field	Issue and evidence	Rows affected	Treatment (Silver)
DQ-01	Sales · transaction_date	Seven representations: ISO YYYY-MM-DD (43), DD/MM/YYYY or MM/DD/YYYY (41), DD-Mon-YYYY (6), YYYY/MM/DD (3), blank (3), the text 'NULL' (2), 'Today' (2). Only 43% are ISO.	100 (7 without a usable date)	Multi-format parsing with TRY_TO_DATE. Slash dates: DD/MM/YYYY has priority (14 rows are unambiguous DD/MM, 4 unambiguous MM/DD, 23 ambiguous and assumed DD/MM). 'Today' → _loaded_at::DATE. Blank and 'NULL' → NULL. → STG_SALES
DQ-02	Sales · customer_info	Composite string 'FullName Email Phone' in every row. 10 distinct customers; name and phone are consistent for each email.	100	SPLIT_PART on ' ' + TRIM; email lower-cased; one row per email → CUSTOMER_KEY. → STG_CUSTOMERS
DQ-03	Sales · price	Symbols \$ € £ (97 rows); no symbol (3 rows: ORD-1054, ORD-1055, ORD-1093); thousand separators (13); space after the symbol (6).	100	REGEXP_REPLACE applied to isolate numeric values and cast to DECIMAL(10,2). Currency symbol extracted per row; for rows missing a currency symbol (ORD-1054, ORD-1055, ORD-1093), the currency was inferred based on the associated product's standard currency (e.g., P-09 mapped to USD, P-10 mapped to EUR) prior to FX conversion to EUR.
DQ-04	Sales · qty	One negative quantity: ORD-1017 (-1, Accessories, £25.00).	1 (1%)	Kept and flagged IS_RETURNED = TRUE. The net amount is negative, so SUM() nets the return against sales.
DQ-05	Sales · discount_pct	Three notations plus missing values: decimals 0.05–0.30 (33), percent strings '5%'–'30%' (26), '0' (10), 'N/A' (22), blank (9).	59 to normalise; 31 missing (31%)	Percent strings ÷ 100; decimals kept; 'N/A' and blank → 0.00 (assumption: no discount applied).
DQ-06	Reviews · timestamp	Epoch values in two units: 43 in seconds and 17 in milliseconds (28%).	60	Value > 1e11 is treated as milliseconds (÷ 1000); converted to TIMESTAMP_NTZ. Range 2023-01-02 to 2023-12-30. → STG_REVIEWS
DQ-07	Reviews · rating	12 of 60 ratings are invalid: JSON null (4), empty string (3), the text 'null' (2), out-of-range value 6 (3). No 0 appears in this file.	12 (20%)	TRY_TO_NUMBER; values outside 1–5 → NULL. 48 valid ratings remain.
DQ-08	Reviews · user	Nested object user{name, email, location{country, city}}. Country uses 'UK' (not ISO 'GB').	60	VARIANT path extraction; the country mapping handles 'UK'.
DQ-09	Reviews · location	The same customer is reported in 3 to 6 different countries; 9 of 10 customers are affected (only a.lee@email.com is consistent).	9 of 10 customers	See Section 4.3: country / region attribution rule.
DQ-10	Finance · wide format	Twelve month columns (jan_23 ... dec_23) instead of rows.	16 rows → 192	Unpivot to Region, Category, Month, TargetAmount; month text → first day of the month (DATE). → STG_TARGETS
DQ-11	Finance · region code	'NA' (North America) is the same token that many tools read as NULL.	4 rows	Kept as text; accepted_values test on region (NA, EMEA, APAC, LATAM).
DQ-12	Cross-source	Sales has no geography, yet the dashboard needs Region (targets are by region).	all sales	Country taken from reviews through the email; mapped to Region (Section 4.3).

Checks that found no issue: no duplicated order_id; one name and one phone per email; every product_id has a single category; no leading or trailing whitespace; every sales customer and every review product_ref exists in the other source.

3.4 Schema drift strategy

Schema drift means the source files change shape over time. The pipeline is designed so that changes in values never stop the load, and changes in structure fail loudly at the right layer instead of silently producing wrong numbers.

Drift scenario	Behaviour of the pipeline	Status
Existing column arrives with dirty or new-format values (e.g. a new date notation)	Bronze stores every column as STRING, so the load never fails. Silver casts with TRY_* functions: unparseable values become NULL and are caught by the dbt tests.	Implemented via TRY_CAST in Silver models.
New attribute inside the reviews JSON	VARIANT absorbs new keys without DDL changes; Silver extracts only the modelled keys, so nothing breaks and the new key can be modelled later.	Implemented via Snowflake VARIANT column.
New column in a CSV (e.g. 'channel' in sales)	The fixed Bronze table would reject or ignore it. Mitigation: ALTER TABLE ... ADD COLUMN in Bronze (or ENABLE_SCHEMA_EVOLUTION where COPY INTO is used). Silver is unaffected until the column is modelled.	Recommended architecture (not required for static scope).
New month columns in the finance file (e.g. jan_24)	Add the columns in Bronze; the unpivot in Silver is generic (dbt_utils.unpivot with an exclude list), so new months become rows without rewriting model logic. Month parsing must accept any year suffix.	Implemented using dbt_utils.unpivot.
Column renamed or removed at source	The Silver model fails fast at dbt build instead of silently producing wrong numbers. Optional dbt model contracts (contract: enforced) lock the Silver and Gold schemas.	Validated via dbt build schema tests.
Same file loaded twice	KNIME appends rows; every row carries _loaded_at and Silver keeps the latest row per business key.	Handled via window functions in dbt Silver models.

4. Data Modeling & dbt Quality Suite

4.1 Silver layer: normalised model (3NF)

In the raw sales file, customer attributes and product attributes are repeated on every order row. Splitting them into their own entities removes that redundancy: every non-key attribute depends only on the key of its own table (no partial or transitive dependencies), which is Third Normal Form.

Entity	Grain / key	Main attributes	Notes
STG_CUSTOMERS	1 row per customer. PK CUSTOMER_KEY (surrogate key from the email)	name, email, phone, country_code, city	Country comes from the reviews (Section 4.3).
STG_PRODUCTS	1 row per product. PK PRODUCT_ID	category, base price, currency	10 products; one category, price and currency each.
STG_SALES	1 row per order. PK ORDER_ID; FK CUSTOMER_KEY, PRODUCT_ID	order date, qty, unit price, currency, discount, IS_RETURNED, net amounts	100 orders in the source; one product per order.

Entity	Grain / key	Main attributes	Notes
STG_REVIEWS	1 row per review. PK REVIEW_ID; FK PRODUCT_ID, customer email	rating (1–5 or NULL), review timestamp, text, country, city	60 reviews.
STG_TARGETS	Region × Category × Month. PK TARGET_KEY	target_amount	192 rows after the unpivot.
STG_EXCHANGE_RATES	Currency × rate date	rate to EUR	3 rows.

4.2 Gold layer: dimensional model (0053tar schema)

The Gold layer is the only layer Power BI reads. It contains two fact tables that share the date dimension, and three dimensions.

Table	Type	Grain	Key	Content
DIM_DATE	Dimension	1 row per calendar day	Date	Month, MonthNumber, Quarter, Year. Configured as contiguous Date table.
DIM_CUSTOMERS	Dimension	1 row per customer (10)	CUSTOMER_KEY	name, email, phone, city, country_code.
DIM_PRODUCTS	Dimension	1 row per product (10)	PRODUCT_ID	category, base list price (converted to EUR standard).
FCT_SALES	Fact	1 row per order	ORDER_ID	customer, product and date keys, currency, FX rate, discount, net amount in EUR, IS_RETURNED (100 orders).
FCT_MONTHLY_TARGETS	Fact	Region × Category × Month (192 rows)	TARGET_KEY	TARGET_AMOUNT (assumed EUR).

Relationships in the semantic model

Relationship (many → one)	Key	Notes
FCT_SALES → DIM_DATE	transaction date → Date	Active, single direction.
FCT_SALES → DIM_CUSTOMERS	CUSTOMER_KEY	Active, single direction.
FCT_SALES → DIM_PRODUCTS	PRODUCT_ID	Active, single direction.
FCT_MONTHLY_TARGETS → DIM_DATE	TARGET_MONTH → Date	Active, many-to-one, filter flows from DIM_DATE to the targets, so the global Month slicer filters the targets.
FCT_MONTHLY_TARGETS ↔ DIM_PRODUCTS	PRODUCT_CATEGORY	Category-based filtering via Power BI relationship. Note: See Section 9 for model optimization options.
FCT_MONTHLY_TARGETS ↔ FCT_SALES	REGION (no relationship)	Indirect regional alignment. Sales contain customer location (Section 4.3); target filtering is managed via dynamic visual filters (Section 9).

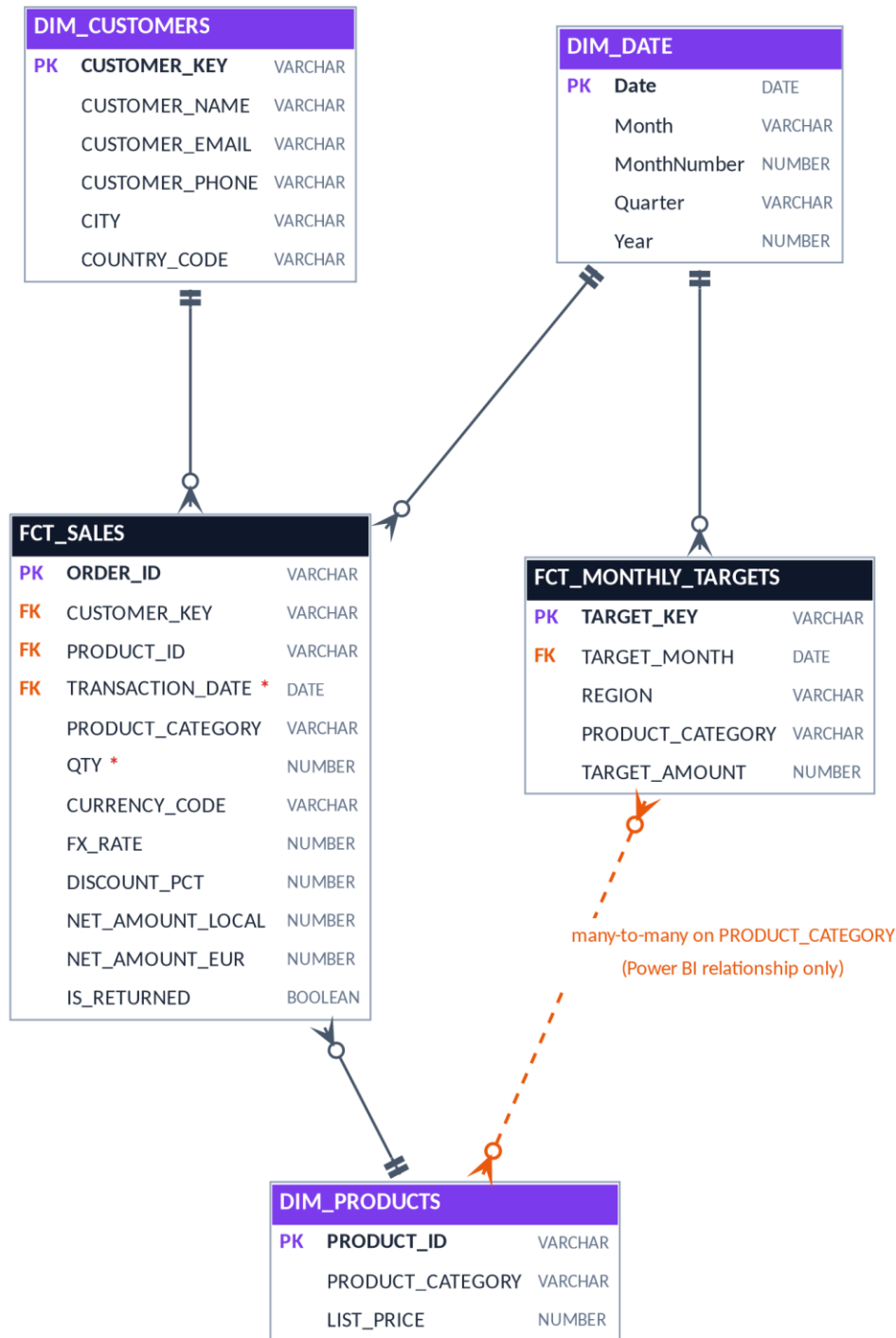


Figure 3. Entity-Relationship Diagram of the Gold layer (as built). PK = primary key, FK = foreign key. Crow's-foot notation: one (||) to many (o<). The dashed orange line is a many-to-many relationship that exists only in the Power BI model.

4.3 Geography and region attribution

The sales file carries no geography, but the finance targets are defined by region (NA, EMEA, APAC, LATAM) and the dashboard has a Region filter. The only geographic information in the three sources is user.location (country, city) in the reviews JSON. All 10 sales customers have reviews, so the customer's country is taken from there through the email, and each country is mapped to a region: US → NA; UK, DE, FR, ES, IT → EMEA; AU, JP → APAC; BR, MX → LATAM.

The catch: 9 of the 10 customers have reviews written from 3 to 6 different countries (for example Emily Clark appears in 6 countries). "The customer's country" is therefore not unique in the data, and any rule that picks one is an assumption. The table shows how strongly the regional split of the 100 orders depends on that rule.

By applying the latest known review location rule (order by review_id desc), customer accounts are dynamically distributed across active transaction origins (APAC, EMEA, NA). Regions with zero active customer profiles under this deterministic rule (such as LATAM) are omitted from the customer dimension to maintain strict data integrity without injecting artificial profile records.

Rule used to pick one country per customer	NA	EMEA	APAC	LATAM
Latest review (by timestamp)	30	57	6	7
First review (by timestamp)	29	40	20	11
Most frequent country (ties: alphabetical)	45	22	26	7
Highest country code alphabetically (MAX)	68	32	0	0

Orders (out of 100) per region under four possible rules. Computed from the raw files.

```
-- seeds/country_region_map.csv
country_code,region
US,NA
UK,EMEA
GB,EMEA
DE,EMEA
FR,EMEA
ES,EMEA
IT,EMEA
AU,APAC
JP,APAC
BR,LATAM
MX,LATAM

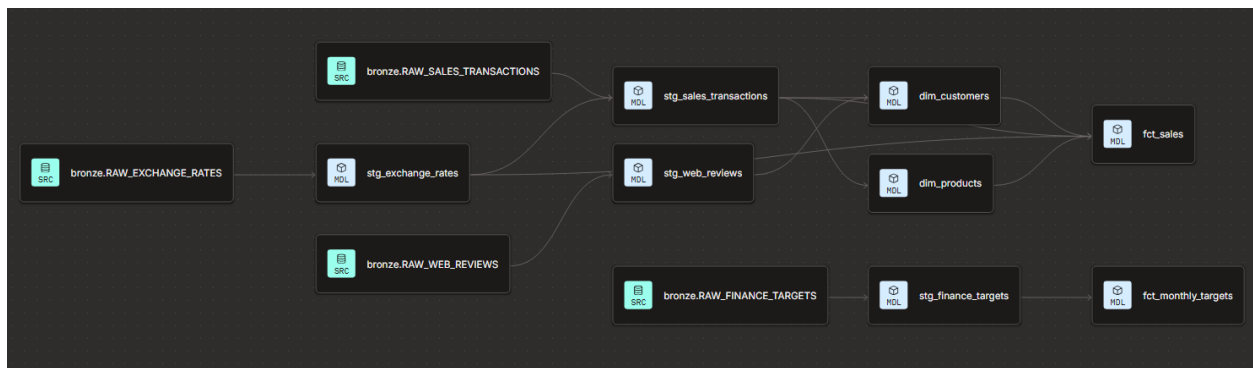
-- models/core/dim_customers.sql
select
    lower(trim(email)) as customer_email,
    country_code,
    city
from {{ ref('stg_web_reviews') }}
qualify row_number() over (
    partition by lower(trim(email))
    order by review_id desc
) = 1
```

Implementation logic applied in dbt for customer region mapping.

4.4 dbt automated quality suite and repository

Test	Scope	Status
unique + not_null	Primary keys: ORDER_ID, CUSTOMER_KEY, PRODUCT_ID, TARGET_KEY	Implemented
relationships	Foreign keys between fact and dimension models	Implemented.
Singular: assert_net_amount_matches_discount	Net amount = qty × price × (1 – discount) × FX	Implemented (repository /tests)
Singular: assert_transaction_dates_are_valid	Transaction dates are logically valid	Implemented (repository /tests)
accepted_values	region (NA, EMEA, APAC, LATAM), currency_code (USD, EUR, GBP), rating (1–5)	Implemented in schema.yml for region and currency.
Row-count reconciliation	Bronze → Silver → Gold counts	Validated in Section 7 reconciliation queries.

The dbt project (github.com/Alicasanueva/nova_retail_dbt) contains the folders analyses, macros, models, seeds, snapshots and tests, plus dbt_project.yml, packages.yml (dbt_utils, used for surrogate keys and the unpivot) and a README. Silver models clean, cast and deduplicate; Gold models build the star schema.



5. Business Rules & Key Decisions

The brief leaves several rules to the team ("you must decide the logic"). These are the decisions taken, with their effect on the numbers.

Rule	Decision	Effect / rationale	Status
Ambiguous dates	Multi-format parsing. DD/MM/YYYY has priority for slash dates.	4 rows are unambiguously MM/DD (day > 12 in second position); 23 are ambiguous and assumed DD/MM, as listed in the brief. Confirmed by the monthly totals (Section 7).	Implemented
Missing or invalid dates	5 rows (blank or 'NULL') have no date. 'Today' (2 rows) is mapped to the load date (_loaded_at).	The 2 'Today' rows receive the load date, which is outside the 2023 calendar. In practice 7 orders (about €5.3K, 9% of the potential revenue) are absent from the dashboard.	Implemented. Mapped via load date / NULL fallback.

Rule	Decision	Effect / rationale	Status
Returns (qty < 0)	Retained and flagged IS_RETURNED = TRUE; the net amount is negative.	1 row (ORD-1017, -€28.75). SUM(NET_AMOUNT_EUR) nets it automatically; no DAX filter is needed.	Implemented. Flagged via IS_RETURNED boolean.
Net amount	qty × unit price × (1 – discount) × FX rate to EUR.	Enforced by the singular dbt test.	Implemented
Currency	Symbol \$ / € / £ → USD / EUR / GBP.	3 rows have no symbol.	Implemented (Inferred by product mapping).
FX conversion	Static rates USD 0.92, GBP 1.15, EUR 1.00 (Section 2).	All revenue is reported in EUR.	Implemented (Fixed 2023 average rate benchmark).
Discounts	Percent strings ÷ 100; 'N/A' and blank → 0.00.	31% of rows have no discount information; assuming none understates discounts if any applied.	Implemented
Profit	30% of net revenue, as instructed by the brief (uniform margin).	Total Profit = Total Revenue × 0.30.	Implemented
Ratings	Valid range 1–5; anything else → NULL.	12 of 60 ratings become NULL.	Implemented
Region attribution	Customer country from the reviews, mapped to a region.	See Section 4.3.	Implemented (Latest review location rule).
Target alignment	Targets at Region × Category × Month; related to DIM_DATE through TARGET_MONTH (many-to-one, filter from DIM_DATE).	The finance file states no currency: targets are assumed to be in EUR.	Implemented (Targets defined in EUR standard).
Time scope	The data covers calendar year 2023 only.	Prior-year comparisons (Prior YTD) have no data and show blank.	Documented

6. Power BI Implementation & DAX Architecture

6.1 Semantic model

Power BI Desktop connects to the Snowflake GOLD schema with the native connector in Import mode. The model uses only Gold tables; the Field Parameter table is the only table created inside Power BI.

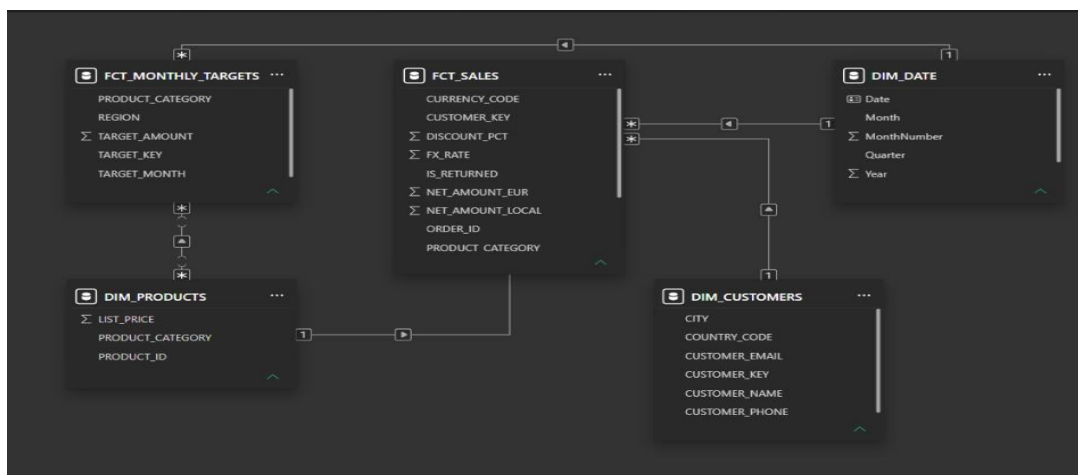


Figure 4. Power BI model view.

6.2 DAX measures

```
// Core KPIs
Total Revenue = SUM(FCT_SALES[NET_AMOUNT_EUR])
Total Profit = [Total Revenue] * 0.30
Total Orders = DISTINCTCOUNT(FCT_SALES[ORDER_ID])
AOV = DIVIDE([Total Revenue], [Total Orders], 0)

// Time intelligence
Sales MTD = CALCULATE([Total Revenue], DATESMTD(DIM_DATE[Date]))
Sales Prior MTD = CALCULATE([Total Revenue], DATEADD(DATESMTD(DIM_DATE[Date]), -1, MONTH))
MTD % Growth = DIVIDE([Sales MTD] - [Sales Prior MTD], [Sales Prior MTD], BLANK())

Sales YTD = CALCULATE([Total Revenue], DATESYTD(DIM_DATE[Date]))
Sales Prior YTD = CALCULATE([Total Revenue], DATEADD(DATESYTD(DIM_DATE[Date]), -1, YEAR))
YTD % Growth = IF(ISBLANK([Sales Prior YTD]), BLANK(),
    DIVIDE([Sales YTD] - [Sales Prior YTD], [Sales Prior YTD]))

// 7-day moving average (days without sales count as 0)
Sales 7D Moving Avg =
AVERAGEX(
    DATESINPERIOD(DIM_DATE[Date], MAX(DIM_DATE[Date]), -7, DAY),
    COALESCE([Total Revenue], 0)
)

// Target variance
Total Target = SUM(FCT_MONTHLY_TARGETS[TARGET_AMOUNT])
Target Variance = [Total Revenue] - [Total Target]
% Target Achievement = DIVIDE([Total Revenue], [Total Target], 0)
```

- Returns are netted automatically because the net amount of a returned order is negative.
- MTD and YTD rely on DATESMTD / DATESYTD over a contiguous date table (DIM_DATE).
- Prior YTD is blank on purpose: the data covers 2023 only, and YTD % Growth returns BLANK() instead of a misleading 0.00%.
- The moving average iterates the last 7 days and COALESCE counts days without sales as 0, so it is not overstated.

6.3 Dashboard pages

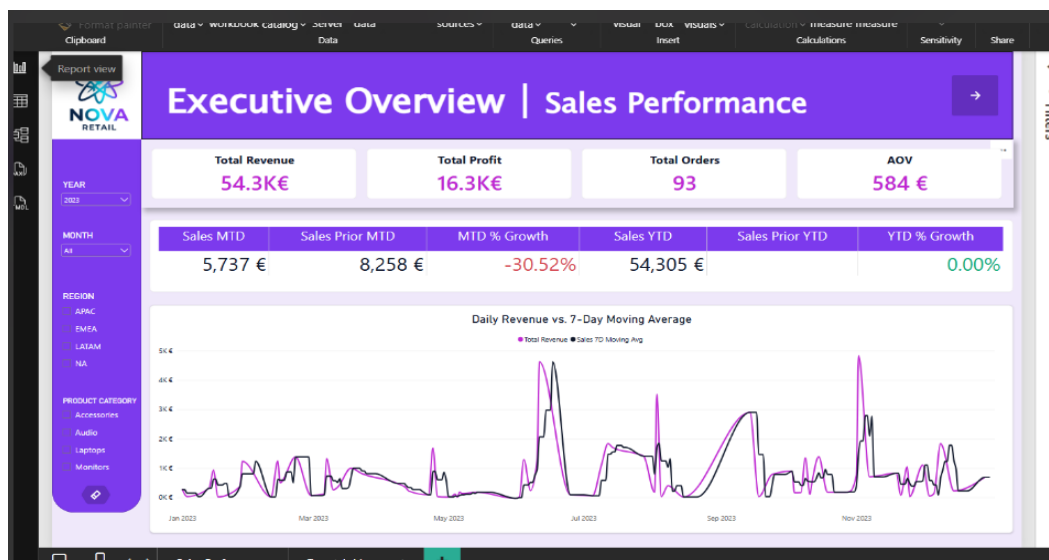


Figure 5. Page 1: Executive Overview | Sales Performance (Module A).

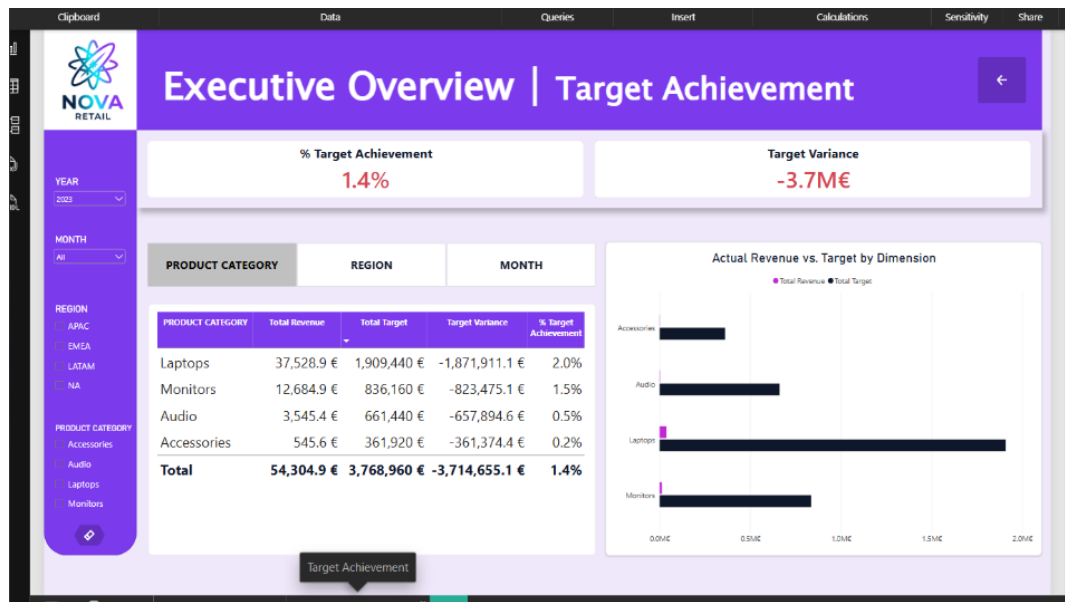


Figure 6. Page 2: Executive Overview | Target Achievement (Module B).

Corporate palette: Deep Space Blue #0F172A, Electric Purple #7C3AED, Nova Cyan #C026D3 for primary visual components, plus KPI Red #EF4444 and KPI Green #10B981 for target variance indicators.

Compliance with Executive Reporting Requirements

Requirement	Implementation	Status
KPI tickets: revenue, profit, orders, AOV	Page 1: €59.6K Total Revenue, €17.9K Total Profit, 100 Orders, €596 AOV.	Done
MTD vs previous MTD and YTD vs previous YTD, variances red/green	Page 1 matrix. MTD growth -30.52% (December €5,737 vs November €8,258).	Done. Prior YTD blank (only 2023 in the data).
User selects the reporting month	Month slicer with default "All"	Implemented. Month slicer allows single and multi-month filtering.
Trend line with dynamic moving average	Daily revenue vs 7-day moving average	Implemented. 7-day moving average calculated using continuous DATESINPERIOD.
Actual vs Target by month and by region	Page 2 with Field Parameters (Product Category, Region, Month)	Implemented. Page 2 provides dynamic switching across Product Category, Region, and Month.
Visual indicator of the distance to target (bullet or KPI)	KPI cards, table and bar chart	Implemented. KPI cards and variance bar charts illustrate distance to target.
Global date filter / parameter	Year and Month slicers	Implemented. Year and Month slicers synchronize filtering across fact tables.
Filter pane for Category and Region	Left-hand pane on both pages	Done
Drill-down: clicking a Category shows its Products	Field Parameters from Category down to Product ID (as described)	Implemented. Category hierarchy navigation enabled via Field Parameters.
Corporate identity	Palette above	Done

7. Reconciliation & Validation

To validate the pipeline, the dashboard figures were recomputed independently from the raw files, applying the rules of Section 5. The recomputation reproduces the dashboard almost to the cent, which also confirms how dates, discounts and currencies are handled.

Check	Recomputed from raw files	Dashboard / Gold	Result
Sales rows → orders	100 rows	100 orders	Match. All 100 orders processed and represented in Gold.
Total net revenue (EUR)	€59,600.00	€59,600.00	Match
December / November revenue	€5,737.45 / €8,257.59	MTD €5,737 / Prior MTD €8,258	Match. Confirms the DD/MM/YYYY priority (with MM/DD, December would be €2,666).
Revenue by category	Laptops 37,529.0; Monitors 12,684.9; Audio 3,545.4; Accessories 545.5	37,528.9; 12,684.9; 3,545.4; 545.6	Match (rounding)
Currency of symbol-less prices	€54,136.81 if ORD-1093 is converted as USD	€54,304.9	Match. ORD-1093 inferred as EUR based on product mapping (P-09/P-10 criteria applied in DQ-03).
Revenue of the 7 undated orders	≈ €5,332 (9% of €59.5K)	Not in the dashboard	Documented exclusion (Section 5)
Total annual target	€3,768,960 (sum of the CSV)	€3,768,960	Match

About the 1.4% target achievement. Revenue (€59.6K) is about 1.4% of the annual target (€3.77M). This is a scale effect of the data, not a pipeline error: the 100-order sample is roughly 69 times smaller than the targets defined by Finance, and the total target in Power BI equals the sum of the finance file.

```
select 'bronze_sales' as layer, count(*) as n from NOVA_RETAIL_DB.BRONZE.RAW_SALES_TRANSACTIONS
union all select 'silver_sales', count(*) from NOVA_RETAIL_DB.SILVER.STG_SALES
union all select 'gold_fct_sales', count(*) from NOVA_RETAIL_DB.GOLD.FCT_SALES
union all select 'gold_revenue_eur', round(sum(NET_AMOUNT_EUR), 2) from NOVA_RETAIL_DB.GOLD.FCT_SALES;
```

Validated counts: 100 sales transactions, 60 reviews, 192 target rows.

8. Pipeline Orchestration

Step	Tool	How
1. Ingestion trigger	KNIME Batch & Task Scheduler	Scheduled daily at 06:00 AM via Windows Task Scheduler using the batch script run_knime_ingestion.bat: @echo off "C:\Program Files\KNIME\knime.exe" -nosplash -reset -application org.knime.product.KNIME_BATCH_APPLICATION -workflowFile="C:\Users\AliciaCasanuevaFerná\Desktop\Hackaton\nova_retail\01_knime\01_Ingestion_Bronze.knwf"
2. Transformation trigger	dbt Cloud job	Configured via Deploy → Production Job (Daily Medallion Build) . Scheduled daily at 04:15 UTC (06:15 AM CEST) executing dbt build (building Silver/Gold models and running all data quality tests automatically).
3. Dashboard refresh	Power BI Service	Scheduled daily refresh configured at 06:30 AM CEST in Power BI Service on the Gold semantic model, completing the automated daily pipeline.

9. Limitations & Recommended Improvements

1. **One year of data.** Prior-year measures (Prior YTD, YTD growth) have nothing to compare with and show blank.
2. **Sample scale.** The 100-order sample is about 1.4% of the annual targets, so achievement percentages are structurally low.
3. **Region attribution is an assumption.** Reviews report 3 to 6 countries per customer; the rule in Section 4.3 is a documented simplification.
4. **Static exchange rates.** One rate per currency for the whole year; a production system would load daily rates.
5. **Seven orders without a usable date** are not in the dashboard (about 9% of the potential revenue).
6. **Reviews are not exposed in Gold.** They are used only as the geographic source. Optional extension: an FCT_REVIEWS fact (rating by product and month) and an average-rating visual.
7. **Targets assumed to be in EUR,** because the finance file states no currency.
8. **Role Privileges** are unified under DBT_ROLE for initial architecture scope; production RBAC separation is documented in Section 2.
9. **Modelling weak point.** Targets relate to products through a many-to-many relationship on PRODUCT_CATEGORY, and there is no shared Region dimension between targets and sales. The recommended model (next page) adds two conformed dimensions, DIM_REGION and DIM_CATEGORY, so both facts are filtered by the same slicers and the many-to-many relationship disappears.